Printer Gutenberg

Spruce version 11.0 -- spooler version 11.0

File: DSK: HAL; 20MAN 1

Creation date: 10 AUG 1980 1927

For: KMP

18 total sheets = 17 pages, 1 copy.

A 6.001 User's Guide to the DECsystem-20

By Kevin B. Theobald

First draft started 3/17/80, completed 4/22/80
Lastest revision 5/8/80

This manual is a brief introduction to the TOPS-20 operating system, intended primarily for use by students in 6.031. The manual is divided into two main sections. The first section outlines the general procedure for writing, editing and running LISP programs on the DECsystem 20, then introduces the TOPS-20 operating system and its role in this procedure. The second section is a simple introduction to the EMACS text editor. The manual is geared toward students who may have had prior computer experience, but are unfamiliar with the systems presented here.

When first learning to use these systems, the beginner should first scan this manual to gain an overall view of how the system works, then read the manual thoroughly while sitting at a terminal. This manual is designed for this method of learning, so there are many examples showing sample computer input and output.

The following symbols, conventions and terms are used throughout this manual:

Symbol or Type	Description
^	This represents the CTRL (CONTROL) key, located directly to the left of the "A" key. The symbol means that the CTRL key should be held down while the character immediately following the "^" symbol is typed. This rule only applies to the first character following the "^". For example, "^XS" represents a "^X" followed by an ordinary "S". Characters generated while the CTRL key is depressed are called control characters. If the computer prints a control character, it will preceed the letter with the "^" symbol. Note that there is a "^" character on the keyboard. Typing this character does not have the same effect as the CTRL key.
{}	Braces indicate that the phrase between them is not a string of characters to be typed in literally, but rather a general term for a variable object. For example, the form delete {filename} means that you should type in "delete" followed by a space and the name of your file.
{cr}	Indicates that the RETURN (carriage return) key, located in the lower-right corner of the keybord, should be typed. TOPS-20 commands are generally terminated with a {cr}, although this is not true in EMACS.
{esc}	Indicates that the ESC key (ESCAPE; ALT or ALTMODE on some terminals), located in the upper-left corner of the keyboard, should be typed. If the computer echos the ESC character (i.e., if it prints it on the screen when you type it), it will use a dollar sign ("$") to designate the ESC. Like the CTRL key, the ESC key is not the same as the "$" key.

{del} Indicates that the DEL key (DELETE; RUB or RUBOUT on some terminals), usually located above the RETURN key, should be typed. When typing, {del} will delete the last character typed.

upper-case None of the systems described in this manual distinguish between upper- and lower-case letters, so unless the user is typing English text, everything can be typed using lower-case letters. In this manual, upper-case letters are used to indicate computer output. For example, the form

```
login{esc} (USER)
```

indicates that the user types login{esc} and the computer responds with (USER).

The only exceptions to this rule are the control characters. Control characters do not distinguish between upper- and lower-case letters, so they are conventionally represented by upper-case letters.

cursor The cursor is a blinking square on the terminal screen. The cursor indicates where the next character typed will be inserted.

1. INTRODUCTION TO TOPS-20

The general procedure for using LISP on the TOPS-20 system is as follows:

1) Log into the TOPS-20 operating system.

2) Invoke EMACS, the text editor.

3) Type in your functions and store them in a file.

4) Get out of EMACS (return to TOPS-20), and invoke the LISP interpreter.

5) Read your file into LISP and test your functions. Then return to TOPS-20. If your functions work, log out of TOPS-20.

6) If your functions don't work, return to EMACS to make corrections to your file. Then return to LISP (going through TOPS-20) and try again. Repeat step 6 until your functions work. Then return to TOPS-20 and log out.

Note the importance of TOPS-20. This is the operating system, the top-level link between you and the computer. EMACS and LISP are accessed through TOPS-20. The system also has commands which allow you to perform "housekeeping" functions with your files, such as listing them and getting rid of those that are old and unneeded.

1.1 LOGGING IN AND OUT

To use TOPS-20, EMACS and LISP, you must first get into EXEC, the part of TOPS-20 that interprets the commands you give. Then you must log into your account.

First, find a free terminal. You should see a message like

WELCOME TO MIT-EECS. HOLD DOWN THE CTRL KEY AND TYPE C TO GET STARTED.

at the bottom of the screen. If the terminal is already on, you should see either the above message or one like

```
LOGOUT JOB 11, USER 031.THEOBALD, ACCOUNT 6031, TTY 6,
  AT 20-MAR-80 19:21:54,  USED 0:1:30 IN 0:50:20
```

If you don't see such a messgae, then the last user probably forgot to log out. Type ^C, then type

```
logout
```

and try again. (Terminate this command and all others in this section with {cr} [carriage return] unless otherwise noted.)

To get the system's attention, type ^C. The system should respond with a short message identifying itself, then print a '@' character. This character is a prompt, which means that EXEC, the command interpreter, is waiting for you to type something.

The format for logging in is

```
login {username} {password}
```

Note that the three words are separated by spaces. Your {username} is the name of your account, and it distinguishes your accounts from others. Your {password} is an access word which only you know. The password prevents others from using your account. In an effort to keep your password secret, the terminal doesn't echo (display your input on the terminal) when you type your password. Therefore, if your username is 031.theobald, the terminal will display

```
@login 031.theobald
```

If you receive a message like

```
?DOES NOT MATCH DIRECTORY OR USER NAME
```

then you didn't type the username properly. Similarly, the message

```
?INCORRECT PASSWORD
```

means you typed the password incorrectly. Try logging in again.

When you are finished using the terminal, return to EXEC (with a ^C) and type

```
logout
```

1.2 SPECIAL TOPS-20 CHARACTERS

There are several special characters you can use to correct typing mistakes and invoke special TOPS-20 functions. These characters, unlike most commands in this section, are not followed by a {cr}.

Character	Description
{del}	This deletes the last character typed. For example, la{del}ogiu{del}{del}out is equivalent to "logout". A video terminal will move the cursor backwards and remove the mistake. A printer will reprint the mistake and follow it with a slash, but the mistake is still removed from EXEC's memory.

^U This deletes the entire line just typed.

^L This retypes the screen.

^Q This is used only if your terminal is in "page" mode (see sec. 1.5). If the terminal has printed a page and has stopped, ^Q will start the printing again. This is explained in further detail in sec. 1.5.

{esc} If this is typed while at the EXEC command lavel, it will invoke a feature called "recognition input". When {esc} is typed, EXEC will attempt to complete a command field if it is incomplete. A field is a part of the command which is separated from the other parts by spaces or other punctuation marks. For example, the login command has three fields: "login", the username and the password. The partially-completed command field must have enough letters to make it indistinguishable from the other possible commands, otherwise, typing {esc} will do nothing but make the terminal "beep". For example, lo{esc} will not work, since there are commands called load, login and logout.

The {esc} command will not complete your password, for obvious reasons, although it will complete your username.

When the computer completes the command field, it will also print a phrase indicating what should go in the following command field. For example, if you type

```
log{esc}
```

the computer will complete the field and ask for the next field:

```
logIN (USER)
```

(Even though both "login" and "logout" begin with "log", you only need to type "log" to log in, since EXEC knows that you are trying to log in. However, you must type "logo" to log out.)

Once the entire command has been completed, a {cr} will tell EXEC to execute it.

Note: you do not need to use the {esc} function to complete a command field. To type a command, you only need to type enough letters to distinguish it from all other commands. For example, you only need to type "dir" to use the "directory" command. Once you are familiar with the commands, you may use this time-saving feature.

^W This deletes the command field currently being typed. For example, if you type

```
login 032
```

followed by a ^W, EXEC will delete the "032"; you may then type in another username.

? This is a call for help, and it can be used in many ways.

Typing ? after a @ prompt will produce a list of all EXEC commands.

Typing ? instead of a command field will produce a list of all the options you can choose to put in the command field.

If {esc} does not complete an incomplete field because the field doesn't have a unique completion, "?" will produce a list of those commands or fields start with your field. For example, if you type

```
lo{esc}
```

the terminal will "beep", because EXEC cannot uniquely complete this command. If you then type "?", EXEC will respond with

```
lo? COMMAND, ONE OF THE FOLLOWING:
LOAD   LOGIN   LOGOUT
@LO
```

Note that your input line is retyped, so you may complete the command as if you had just typed in the first two characters.

1.3 RUNNING PROGRAMS

There are two kinds of input that TOPS-20 will accept: commands and program names. Commands, such as login and logout, are routines executed by EXEC. Programs are routines which exist on a lower level, hence, they are not executed by EXEC, but are only accessed by it.

To invoke a program, type its name to EXEC. Thus, typing "emacs" will invoke EMACS, and "lisp" will invoke LISP. When you invoke a program, you create a "fork", which is an indicator that you are using that program.

When you have finished using a program, you can get back to EXEC by typing ^C^C. (Actually, only one ^C is needed if the program is awaiting input, but two ^Cs will always work and the extra ^C will not harm anything.) If you exit a program in this manner, you will not terminate it; it will only be suspended. The fork you created will remain intact.

To reactivate a suspended program, type its name again. Your program will continue where it left off when you suspended it.

If you follow the steps at the beginning of sec. 1 on testing LISP functions, your session will look something like this:

```
@login 031.theobald
 JOB 17 ON TTY6 24-MAR-80 20:26
 PREVIOUS LOGIN: 21-MAR-80 18:42
@emacs
[KEEPING]
    .
    .     {use of EMACS}
    .
^C^C
@lisp
[KEEPING]
    .
    .     {use of lisp}
    .
^C^C
@emacs
[CONTINUING]
    .
    .
    .
^C^C
@lisp
[CONTINUING]
    .
    .
```

```
.
^C^C
.
.     {repeat until LISP functions work}
.
@logout
```

To get information about the forks you are using, type

```
information fork-status
```

You will see something like

```
    EMACS (1): KEPT, HALT AT 41301, 0:05:21.5
 =} LISP (2): ^C FROM IO WAIT AT 116710, 0:03:34.7
```

This particular response would indicate that you are currently using both EMACS and LISP. TOPS-20 assigns each fork a number, in the order that they are created. The arrow by LISP indicates that LISP is the "current" fork, i.e., the fork used most recently. If you were to continue EMACS, return to EXEC, then type the information command again, the arrow would point to EMACS.

If you are no longer using a fork, you can remove it by typing

```
reset {fork}
```

where {fork} can be either the name or the number assigned to it. If no argument is given for {fork}, then EXEC will remove the current fork. If the current fork is removed, the highest-numbered fork becomes the current fork. If you reset a fork, EXEC will always print the name of the current fork (the new current fork if you reset the old current fork) in parentheses.

If you type

```
keep {fork}
```

the specified fork (or the current fork if none is specified) will be "kept". If the current fork is "kept", you cannot remove it with the reset command unless you also specify the name or number. This is to prevent you from accidently removing wanted programs. EMACS and LISP automatically "keep" themselves when they are called. Thus, even if EMACS is the current fork, you must type

```
reset emacs
```

or

```
reset {emacs' fork-number}
```

to remove it.

1.4 WORKING WITH FILES

1.4.1 TOPS-20 FILES

When you write files using EMACS, they are stored in a directory assigned to your account. A directory is a block of memory with a certain size limitation, which will quickly be exceeded if you leave old, unneeded files in your directory. This section and the next describe the basic "housekeeping" functions of TOPS-20, including those that allow you to remove old files, print and copy files, and list your directory.

Each file must have a filename to distinguish it from other files. The basic format for a filename is

{device}:<{username}>{name}.{type}.{generation number}

The device is the name of the main memory structure in which your directory is located, and the name of the device is always followed by a colon. The username is the name of your account. It is enclosed in angle-brackets. If the first two fields are not specified, then your device and username are used as default values. Since you will usually be working with files in your own directory, the only part you need to worry about is the last part:

{name}.{type}.{generation number}

{name} is a word that describes what is in the file. It can be, for example, the name of a LISP function or a word describing its purpose or contents. The type is another word; it indicates what type of file it is. The LISP interpreter requires you to use "LSP" to show that the file contains LISP functions. Other standard types include ALG for ALGOL and TXT for text.

Generation numbers are positive integers used to specify the revision number of the file. Generally, the first draft of a file is generation number 1, the first revision is number 2, etc. There is a special feature that allows you to dispense with generation numbers altogether. If you do not specify a generation number when you deal with a file, the system will assign one based on the following rules:

1) If the command reads the file, then EXEC looks through all files which match the name and type specified in the command, and chooses the one with the highest generation number. If no file in your directory matches the specified file name and type, an error message is returned.

2) If the command writes the file, then it creates a new file. The generation number of this file is 1 greater than the highest existing generation number. If no file in your direstory matches the specified name and type, the new file is given the generation number 1.

3) Some EXEC commands will act on all files in your directory matching the specified name and type. These commands will be described in sec. 1.6.

For example, suppose your directory is empty. If you create a program in EMACS, use the appropriate EMACS command (see sec. 2) to write it into a file, and you only specify

program.lsp

as the file name, your directory will now contain the file program.lsp.1. If you perform another write function, your directory will contain program.lsp.1 and program.lsp.2. If you read a file, and you don't specify a generation number, the file program.lsp.2 will be read.

If you consistently use this feature, you will always read the most recent revision of your file, and always write revisions into new files. You may then ignore the generation numbers, unless you overcrowd your directory with too many generations; if this happens, you must remove old generations (see sec. 1.4.2).

1.4.2 EXEC COMMANDS

The following section contains a summary of useful "housekeeping" commands. To execute a command, simply type its name. No additional fields are needed unless otherwise specified. The ? and {esc} functions will help you if you aren't sure if additional fields are necessary.

DIRECTORY

This command gives a short listing of the files in your directory. If a file has more than one generation number, all the generation numbers are printed on one line, separated by commas. If a file has more than one type, the additional type names and their corresponding generation numbers are identified and printed below the main file name. For example, the directory listing

```
  PS:<031.THEOBALD>
PROGRAM.LSP.1,2
  .BAS.3
```

indicates that the directory of account 031.THEOBALD, on device PS, has the files PROGRAM.LSP.1, PROGRAM.LSP.2, and PROGRAM.BAS.3.

The commands tdirectory, wdirectory and rdirectory will tell you when your files were written, who wrote them, and how long they are, repectively. The command fdirectory gives all this information in one listing.

TYPE

The command

```
type {filename 1}, {filename 2}, ... {filename n}
```

will give a listing of all the files specified. If you specify only one file, no comma is necessary.

PRINT

This command is identical to the type command, except that the output is sent to the lineprinter instead of the terminal. This is useful if you are using a video terminal and you want a hardcopy (paper) listing of your file. After executing this command, you may type other commands as soon as the prompt reappears, even if the printer hasn't finished printing your file.

DELETE, UNDELETE

The command

```
delete {filename 1}, {filename 2}, ... {filename n}
```

will remove all the specified files from your directory. Any directory listings taken afterward will not list {filename}. However, these files still resides in memory and can be returned to your directory by typing

```
undelete {filename 1}, {filename 2}, ... {filename n}
```

If the generation number is not specified, delete and undelete will affect all generations with the same name and type.

EXPUNGE

Even though deleted files don't appear in your directory, they still reside in memory. You will quickly fill your alloted space in memory if you don't occasionally eliminate old deleted files. The expunge command will permanently flush all deleted files from memory. Once a file has been expunged, it can't be restored, so you should always list your directory before expunging to make sure you haven't deleted anything you want to keep.

COPY

The command

```
copy {filename 1} {filename 2}
```

copies the contents of {filename 1} into {filename 2}; {filename 1} remains intact. The copy command reads {filename 1} and writes onto {filename 2}, so if no generation numbers are specified, the command follows the rules for assigning them, as described in sec. 1.5.

RENAME

The command

```
rename {filename 1} {filename 2}
```

changes the name of {filename 1} to {filename 2}. If generation numbers are not specified, then they are assigned according to the rules described in sec. 1.5.

HELP

Typing

```
help {program name}
```

will describe the program {program name} in detail. Substituting an asterisk (*) for {program name} will print a list of the programs for which help is available.

INFORMATION

This command will provide information about your use of the system. The second field specifies the type of information wanted. For example,

```
information directory
```

gives information about the directory and the amount of memory it is currently using.

```
information fork
```

lists the forks that are in use. (See sec. 1.3.)

```
information terminal
```

lists information about the terminal, such as the type of terminal and the page dimensions.

```
information ?
```

will list all the options available.

TERMINAL

This command has several options that allow you to change some of the terminal's characteristics. For example,

```
terminal width {width}
```

({width} a positive integer) will change the line width to {width} characters.

```
terminal page
```

will change your terminal to page mode. If you use the type command to print a long file, the printing will stop after a page has been printed. To resume the printing, type ^Q.

```
terminal length {length}
```

({length} a positive integer) will change the page length to {length} lines.

TOPS-20 has correct default values for parameters such as the width and length. Therefore, you generally will not need to change any parameters, unless someone has previously tampered with them.

SET DIRECTORY PASSWORD

This command is used to change the password of your account. The format (including computer output) is

```
set directory password <{username}>{cr}
OLD PASSWORD: {old password}{cr}
NEW PASSWORD: {new password}{cr}
RETYPE NEW PASSWORD: {new password}{cr}
```

The passwords are not echoed by the computer.

2. INTRODUCTION TO EMACS

EMACS is the text editor which allows you to write, edit and correct your files. It is a real-time editor, which means that any changes you make in your file are immediately displayed on the terminal screen. This section describes how to write and modify files using EMACS.

2.1 BUFFER STORAGE

The files in your account are stored on bulk storage devices such as magnetic disks. These devices have large storage capacities, but they are too slow to be practical for a real-time editor such as EMACS. The computer has faster memory called random-access memory (RAM). Any location in RAM can be directly accessed by the computer (hence the name), making it ideal for a text editor. However, there is not enough RAM to store a lot of files, so the bulk memory must be used for storing files.

For this reason, EMACS uses a buffer, which is a portion of the RAM used to temporarily hold your file. When you give commands to EMACS, it rapidly makes the changes to the RAM. When you have finished editing your file, you must tell EMACS to save your file. It will then transfer your file to bulk memory.

2.1.1 THE BUFFER POINTER

You can add text to the buffer by typing it in. As you type, the screen will continually update the screen. It is this feature that makes EMACS a real-time editor.

If adding text were all EMACS could do, it would not be very useful. Because the buffer is stored in RAM, it can be accessed at any point. EMACS takes advantage of this property, allowing the user to alter the contents of the buffer to, for example, correct typing mistakes, or change words or

sentences.

EMACS uses a "buffer pointer" to mark the point in the buffer it is working on. The buffer pointer is represented on the screen by the cursor. The cursor always points to a character or space on the screen, but the buffer pointer is actually between the cursor and the previous character. The character at the cursor is "to the right" of the pointer, while the previous character is "to the left". If you type in text, the pointer will always be to the right of the last character typed.

When you first invoke EMACS, there is nothing in the buffer, so the pointer is in the upper-left corner. If you now type

```
(defun fact (n)
```

(with no {cr}) the screen will look like this:

```
(defun fact (n)_
```

(The underline represents the cursor.) If you now type {cr}, the screen will look like this:

```
(defun fact (n)
_
```

EMACS has moved the pointer to the beginning of the next line. EMACS has marked the end of the first line with a {cr}, although the terminal doesn't show it. If you type ^B, the EMACS command to move the buffer pointer back a character, the pointer will move back to the end of the first line.

If the {cr} character is deleted, the following line will be added to the end of the first line. For example, if the screen displays

```
(defun fact (n)_
       (cond ((zerop n) 1)
```

and you type ^D, the command to delete the character pointed to by the pointer, the screen will now display:

```
(defun fact (n)_      (cond ((zerop n) 1)
```

2.2 EMACS COMMANDS

2.2.1 POINTER-DEPENDENT COMMANDS

The following commands are pointer-dependent; they operate on characters near the buffer pointer. These commands, and all others given in section 2, are not terminated with a {cr} unless otherwise noted.

Type of command	Command	Description
MOVING THE BUFFER POINTER	^F	Move the pointer forward one character.
	^B or ^H	Move the pointer backward one character.
	^A	Move the pointer to the beginning of the line.
	^E	Move the pointer to the end of the line.
	^N	Move the pointer to the next line.
	^P	Move the pointer to the previous line.

{esc}F Move the pointer forward until it reaches the end of a word.

{esc}B Move the pointer back until it reaches the beginning of a word.

^Z^A Move the pointer back until it either reaches the beginning of the buffer or the beginning of a DEFUN. EMACS defines a DEFUN as a line that begins with a "(". If the "(" is indented (preceded by spaces or tabs), the line is not a DEFUN.

^Z^E Move the pointer forward until it passes the end of a DEFUN. The end of a DEFUN is the {cr} to the right of the closing delimiter (right parenthesis, bracket or brace) that corresponds to the beginning of a DEFUN. This command should only be used when editing LISP files, since typing ^Z^E when there are no DEFUNs will result in an error message.

For example, suppose the buffer contains the DEFUN

```
(defun print-x (x)
       (print x))
```

and the pointer is at the end of the first line. Typing ^Z^A will move the pointer to the beginning of the first line, while ^Z^E will move the pointer to the beginning of the third line.

^S This command puts EMACS into a search mode. The format for a search is ^S{string}. As you type {string}, EMACS moves the pointer forward until it reaches the end of an occurance of the accumulated string. The pointer is moved to the right of the first occurance. The last character in {string} can be removed by typing {del}.

While you are in the search mode, EMACS will remind you of what you are searching for, by displaying

```
I-SEARCH: {string}
```

in the lower-left corner of the screen.

If there are no occurances of {string} to the right of the pointer, EMACS responds with the message

```
FAILING I-SEARCH: {string}
```

and does not move the pointer. At this point, typing ^G will remove characters from the right end of {string} until there is an occurance of string in the buffer.

Once a successful search has been completed, typing {esc} will cause EMACS to leave the search mode, while ^G will cause EMACS to return the pointer to its original location and quit the search mode. Typing ^S will move the pointer to the next occurance of {string}, and ^R will change the forward search to a reverse search (see below). Typing any other EMACS command will cause EMACS to leave the search mode and execute the command.

If {string} is empty and you type ^S, the string used in the last search becomes the new string. (If this is the first search, EMACS will not do anything.) When you leave a search, EMACS remembers {string}.

^R — This command is similar to ^S, except that it performs a reverse search. The pointer is moved back to the left of the first occurance of {string}.

When in either search mode, you may switch to the other by typing its command (either ^R or ^S).

INSERTING AND DELETING TEXT

{character} — Any ordinary letter, number or punctuation mark is inserted to the right of the pointer. Everything to the right of the new character is shifted to the right to accomidate the new character. The pointer is moved to the right of the new character. For example, to insert an "m" into the word "terinal", you must move the pointer until the cursor points to the "i", then type "m". The buffer will now contain "terminal" and the cursor will still point to "i", although it is now one character to the right of its original position.

^D — The character to the right of the pointer is deleted from the buffer. Everything in the same line to the right of the deleted character is shifted to the left. The cursor remains at the same spot, although it now points to the character to the right of the deleted character.

{del} — This is identical to ^D, except that the character to the left of the pointer is deleted. If the cursor is at the beginning of the line, the {cr} of the previous line is deleted, causing the pointer's line to be added to the end of the previous line.

^K — This command "kills" (deletes) everything between the pointer and the first {cr} (exclusively) to the right of the pointer. If the first character to the right of the pointer is a {cr}, just that {cr} is killed.

^@ — This is used in conjunction with ^W (see below). This command marks the pointer's current location in the buffer.

^W — This command kills everything, including {cr}'s, between the pointer and the spot marked by the ^@.

^Y — This command "yanks" (inserts) the last thing killed (with either a ^K or a ^W) to the right of the pointer. The pointer is moved to the right of the inserted string. This command, in conjunction with ^K or ^W, is useful for moving blocks of text around the buffer by killing a part of the text, moving the pointer, then yanking the text.

^Q — The character immediately following the ^Q is inserted, regardless of what it is. For example, ^Q^L inserts ^L into the text. Although the ^L is printed as two characters, it is actually only one.

SPECIAL COMMANDS

^U — This command allows you to tell EMACS to perform a command a given number of times. The format is

```
^U{number}{command}
```

EMACS will perform the command {command} {number} times. For example, ^U5^F will move the pointer forward five characters. If no number is given, a default value of 4 is given. A ^U-modified command is itself a command, so ^U can be used on itself multiplicatively. For example, all the following move the pointer forward 64 times:

```
^U64^F
^U8^U8^F
^U^U16^F
^U^U^U^F
```

{esc} — When followed by a number, {esc} will have the same result as ^U. However, it can't be used on itself (like ^U8^U8^F), and it doesn't assign a default number if no number is given. Many commands, including {esc}F and {esc}B, begin with the {esc} character.

2.2.2 BUFFER STORAGE COMMANDS

After invoking EMACS from EXEC (see sec. 1.3) you may begin inserting text by simply typing it. However, if you want to modify an existing file, you must load the file from the bulk memory into the buffer. To do this, type

```
^X^R {filename}{cr}
```

If no generation number is given, ^X^R follows the rules for reading files described in sec. 1.4.

The bottom of the screen should display some information, including the word "Main:" followed by your filename. This filename is now the default filename and remains so until a new file is read. If you now type ^X^R with no filename, EMACS will use the default filename.

The command ^X^S will load the buffer into the default file IF the buffer has been changed since being read. If the buffer has not been changed, ^X^S will not perform the command. This command follows the rules for saving files described in sec. 1.4 and 1.5. For example, if you type

```
^X^R program.lis{cr}
```

and the current generation number is 5, then program.lis.5 will be read into the buffer. If the buffer is then changed, typing ^X^S will transfer the buffer to program.lis.6.

The command

```
^X^W {different filename}{cr}
```

will write the buffer onto {different filename}. This command also follows the rules for saving files.

Neither ^X^W nor ^X^S erase the buffer.

2.2.3 OTHER GLOBAL COMMANDS

The following commands do not depend on the location of the pointer:

Command	Description
^C	Suspend EMACS and return to EXEC. If EMACS is executing a command, two ^C's are necessary.
^L	Clear the screen and redisplay the buffer.
{esc}?	Describe the command immediately following the question mark. For example, {esc}?^D describes the delete command. The description is displayed on the screen, but the buffer is unchanged. The next character typed will remove the description and redisplay the buffer. This character will be treated as a command, to be executed by EMACS, so if you want to redisplay the buffer without executing an unwanted command, type ^L.

2.2.4 MM COMMANDS

Typing {esc}{esc} will put you in a mini-buffer, which occupies the top of the screen. When you are in the mini-buffer, typing standard EMACS commands from sec. 2.2.1 will edit the mini-buffer, while leaving the main buffer unchanged. To return to the main buffer, type {esc}{esc}.

The mini-buffer may be used to execute special commands known as MM commands. The format for an MM command is

mm {command name}{esc}{arguments separated by {esc}'s}

The format for executing MM commands is

{MM command 1}{esc}{MM command 2}...{MM command n}{esc}{esc}

None of the commands are executed until the double {esc} is typed. The double {esc} returns EMACS to the main buffer after execution.

The following list describes the more useful MM commands:

mm replace{esc}{string 1}{esc}{string 2}

replace all occurances of {string 1} completely to the right of the pointer with {string 2}.

mm query replace{esc}{string 1}{esc}{string 2}

move pointer to the right of the first occurance of {string 1} and wait for a response:

Response	Result
{space}	Replace this occurance of {string 1} with {string 2} and move to the next occurance of {string 1}.
{del}	Move to the next occurance without replacing this one.
{period}	Replace this occurance and escape the MM command.
{esc}	Escape the MM command without replacing this occurance.

! Replace all remaining occurances without asking.

All replacements follow the same rules used by replace for upper- or lower-case letters. For a complete list of responses, use the MM describe command (see below) to display a complete description.

mm describe{esc}{command name}

describe {command name} (e.g., mm describe{esc}query replace), much like {esc}?, but for MM commands.

mm occur{esc}{string}

list all lines containing {string}.

mm apropos{esc}{string}

list and briefly describe all commands (MM and others) whose names contain {string}.

mm lisp mode

change the EMACS mode from fundamental to LISP (see sec. 2.3).

2.3 LISP MODE

EMACS has several features which make LISP editing easier. The ^Z^A and ^Z^E commands (see sec. 2.2.1) enable you to scan through DEFUNs quickly. Another feature involves the closing of delimiters. When you type a closing delimiter (right parenthesis, bracket or brace), EMACS will search back from the pointer until it finds the corresponding opening (left) delimiter. If the opening delimiter is on the screen at the time the closing delimiter is typed, the cursor will momentarily point to the opening delimiter, although the pointer is unchanged. This allows you to see whether or not you have the right arrangement of parentheses in a LISP function. Note: this feature does not distinguish between parentheses, brackets or braces.

When you invoke EMACS, the bottom of the screen will display a message like

EMACS (Fundamental)

This shows that EMACS is in the fundamental mode. You can change this to the LISP mode by typing

{esc}{esc}mm lisp mode{esc}{esc}

(see sec. 2.2.4). Now the screen will read

EMACS (LISP)

When in the LISP mode, you can use the tab key to invoke a LISP pretty-printing function. The pretty-printing routine will reposition the current line in accordance to certain rules for properly indenting LISP functions. For example, if the buffer contains

```
(defun fact (n)
(cond ((zerop n) 1)
(t (fact (sub1 n)))))
```

and the pointer is in the second line, typing {tab} will change the buffer to this:

```
(defun fact (n)
```

```
(cond ((zerop n) 1)
      (t (fact (sub1 n)))))
```

For information about the rules of pretty-printing, see LISP documentation.